

AI-BASED GATEMAN SYSTEM To Record Vehicle Number Plates

SOMNATH BERA

An earlier version of this automatic gatemanager system, built around a camera-based design, was published on the Electronics For You website and can be accessed here:

<https://www.electronicshobby.com/electronics-projects/automatic-gatemanager>

That system used an ultrasonic distance sensor to detect approaching vehicles, after which a Raspberry Pi camera captured the number plate image. While the concept worked, its performance revealed several practical limitations. The ultrasonic sensor could not reliably distinguish between a vehicle and a person, so anyone carrying a fake number plate could unintentionally trigger the capture process. Recognition accuracy was inconsistent, and overall operation remained slow.

To address these shortcomings, a more advanced and robust architecture has been developed. In the updated system, the camera not only captures and stores the vehicle's number plate in a database but also runs an additional machine-learning model in parallel to classify the approaching object.

This enables clear differentiation between vehicles, humans, animals (such as cats and dogs), fake boards, and counterfeit number

```
<untitled> *
1 import subprocess
2 import cv2
3 import easyocr
4 import os
5 from ultralytics import YOLO
6 import threading
7 import time
8 import logging
9 from datetime import datetime
10
11 # === Setup ===
12 image_path = "test.jpg"
13 output_car_dir = "output/cars"
14 output_plate_dir = "output/plates"
15 os.makedirs(output_car_dir, exist_ok=True)
16 os.makedirs(output_plate_dir, exist_ok=True)
17
18 # === Logging ===
19 log_file = "detection_log.txt"
20 logging.basicConfig(filename=log_file, level=logging.INFO, format='%(asctime)s - %(message)s')
21 def log_output(msg): logging.info(msg)
22
23 # === Models ===
24 reader = easyocr.Reader(['en'], gpu=False)
25 car_model = YOLO('../resources/yolov8n.pt')
26 plate_model = YOLO('../resources/kbest.pt')
27
28 # === Image Capture ===
29 def capture_image():
30     """Capture image using libcamera-still quickly."""
31     try:
32         subprocess.run(['libcamera-still', '-t', '100', '-n', '-o', image_path], check=True)
33         log_output(f"Image captured: {image_path}")
34     except subprocess.CalledProcessError:
35         log_output("Image capture failed.")
36
37 # === OCR Job ===
38 def ocr_job(gray_crop, car_i, plate_i):
39     ocr_result = reader.readtext(gray_crop)
40     if ocr_result:
41         for _, text, conf in ocr_result:
42             clean_text = text.replace(" ", "")
43             log_output(f"{{car_i}}-{{plate_i}} Plate: {{clean_text}} ({{conf*100:.1f}}%)")
44     else:
45         log_output(f"{{car_i}}-{{plate_i}} No text detected.")
46
47 # === Detection Pipeline ===
48 def detect_cars_and_license_plates(img):
```

Fig. 1: Code path configuration

plates. The system first confirms that the detected object is a vehicle and only then proceeds to locate and recognise the number plate.

A YOLO-based object-detection framework supports this process. Two models, YOLO and kbest.pt, operate simultaneously to detect the vehicle and subsequently extract and recognise its number plate with higher accuracy.

Together, these enhancements improve robustness, reliability, and operational efficiency. The system runs on a Raspberry Pi 5 (or Raspberry Pi 4), selected for its strong processing capability and suitability

for edge-based AI applications. Continuous YOLOv8 inference generates considerable heat, making effective thermal management essential.

To manage this, a cost-effective active cooling solution has been integrated. The setup uses a forced-air cooler with an onboard temperature and RPM sensor connected directly to the Raspberry Pi 5. This arrangement enables real-time thermal monitoring and maintains stable operating conditions during extended AI workloads.

With active cooling in place, the Raspberry Pi operates within safe temperature limits and supports slight

BILL OF MATERIALS

Components	Quantity
Raspberry Pi 5 (8GB)	1
MicroSD card (32GB or larger)	1
Power adaptor 5V DC, 3A	1
Raspberry Pi camera	1
RPi camera ribbon cable connector	1

EFY note: Apart from the above components, an HDMI monitor, keyboard, and mouse are also required.